

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_e1b943dd-dac\agent-a821abf1fd3e2b7f7.jsonl`  
- SHA-256: `0e397013bbfbc71d9892f53573b7795a0c1c0b804bdcd2060cb000d8c9c46be8`  
- Source modified: `2026-07-06T19:19:34+00:00`  
- Imported at: `2026-07-08T16:00:12+00:00`  
- project: `wf_e1b943dd-dac`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T19:18:19.572Z)

Reviewing GitHub PR #40 of the repo at C:/Users/avidu/Projects/Telegram ? a DRAFT, DOCS-ONLY PR (branch docs/track-f-policy-pipeline).
It adds ONE file: docs/adr/0012-policy-pipeline.md (ADR 0012, status Proposed) ? the Track F design doc for a four-tier "policy pipeline" (Tier 1 metadata gates / Tier 2 topic preferences / Tier 3 content safety / Tier 4 deep inspection), a PolicyStage protocol + StageResult explanation trace, and 7 phased follow-up issues F1-F7.
No code changes. Read the ADR:
git -C C:/Users/avidu/Projects/Telegram show docs/track-f-policy-pipeline:docs/adr/0012-policy-pipeline.md

This is a DESIGN review, not a code review ? there is no gate to run. Judge the ADR as a design contract:
soundness, completeness, internal consistency, accuracy of its claims, and consistency with the codebase and the other ADRs (docs/adr/0001..0011 exist on main; read any you need via 'git -C C:/Users/avidu/Projects/Telegram show main:docs/adr/<file>').
Relevant current code (on main): src/tg_video_filter/models.py (FilterConfig, ContentItem, Source, Policy),
src/tg_video_filter/db.py (load_filter_config/save_filter_config shims, sources.watched gate should_ingest_source, delivery_attempts/record_delivery), src/tg_video_filter/telethon_client.py (the live handler: watched gate -> dedup -> FilterEngine.evaluate -> insert -> deliver). Track E (sources.watched opt-in gate) and Track C (delivery_attempts) just merged.

Ground rules:

- READ actual files, cite ADR section / file:line. You MAY run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe to check claims (e.g. does FilterConfig ignore/drop unknown JSON keys? does ContentItem carry caption text? is content_item.text populated?).
- This is a DRAFT asking for design feedback ? frame findings as design gaps/risks/inaccuracies, not merge-blockers.
Severity: high (a claim that is false, or a design gap that would break an F-phase / cause data loss), medium (unspecified interaction, inconsistency, missing dependency), low (wording, invalid example, minor).
- Report only issues in THIS ADR. No praise. Be concrete: what's wrong, why it matters for implementation.

ADVERSARIALLY VERIFY this design finding ? try to REFUTE it. Read the cited ADR section + any referenced ADR/code,
run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe checks if useful. CONFIRMED if it genuinely holds against THIS ADR (a real gap/inaccuracy/inconsistency).
REFUTED if the ADR actually addresses it / the claim is wrong / it's out of scope for a design doc. PLAUSIBLE if unsettled. Corrected severity (remember: draft design doc, so calibrate ? false claim/data-loss = high, gap = medium).
FINDING (design): AND-only short-circuit pipeline cannot express include/exclude topic semantics or OR/priority rules ? expressiveness gap not acknowledged
severity medium | section Decision ? Stage model (AND-only short-circuit) vs Topic preferences (include/exclude, mode any/all)
summary: The pipeline is pure AND with first-failure short-circuit, but Tier 2 topics define include (OR/any-or-all) and exclude sets, and Tier 3 has an allowlist ? these are not expressible as an ordered AND chain of independent pass/fail stages without smuggling boolean logic inside a single stage.
why_it_matters: 'mode: any' means pass if ANY include topic matches (OR), which a chain of per-topic AND stages cannot represent; it must collapse into one monolithic topic stage that does its own internal OR ? at which point the 'ordered list of stages' abstraction isn't buying the composability the ADR claims (independent test/reorder/add). Exclude-then-include ordering and allowlist-vs-topic interactions (does allowlist bypass topics? does exclude override include?) are undefined. The Negative section admits interface awkwardness but not the boolean-expressiveness limit.
evidence: evaluate_pipeline: `if not result.passed: return False` ? strict AND, no OR/veto/score. topics config: include + exclude + mode 'any'/'all'. Tier 3 allowlist + blocklist. No rule for how exclude interacts with include, or allowlist with topics.
rec: Acknowledge the AND-only limitation and specify how within-tier boolean logic (include OR, exclude veto, allowlist bypass) is handled ? either as documented single composite stages per tier, or by giving StageResult a richer verdict (e.g. accept/reject/neutral) than a bare bool. State the precedence order between exclude, include, and allowlist.

2. assistant (2026-07-06T19:18:23.280Z)

I'll verify this finding by reading the ADR and checking the pipeline model against the topic/tier semantics it describes.

3. user (2026-07-06T19:18:25.428Z)

ADR 0012: Policy pipeline ? explainable, tiered, user-customizable content gates

- **Status**: Proposed
- **Date**: 2026-07-06
- **Decision owner**: project owner
- **Tracking issue**: #31 (Track F of the provider-neutral umbrella, #25)
- **Depends on**: ADR 0011 (Track B ? domain schema, `policies` table)

Context

The system today evaluates every matched video against a single `FilterConfig` (per account, via `policies.config.json`). The filter is **flat**: all rules run unconditionally, the output is a boolean (`matched`), and there is no way to express topic preferences, security policy, or why a specific video passed or failed.

Track F defines a **policy pipeline** ? an ordered sequence of stages that run before delivery ? so the system can express richer decisions, explain them, and defer expensive work until after cheap rejections.

What we have today (post-Track-E)

Concept	Current state
FilterConfig / Policy	One JSON blob per account. Flat duration/resolution/size/exclude

```

rules. AND semantics. |
| `FilterEngine` | Evaluates all rules; returns `FilterResult(passed, reasons)`. |
| `policies` table | One row per account (is_default=1). `config_json` is the `FilterConfig` blob. |
| Security / topic / deep-inspect | Nothing. Codec/keyword/MIME filters and ffprobe were deferred to v2 in the original plan. |

```

What we want

A **policy** should be a pipeline of ordered **stages**, each with a well-defined cost, inputs, and explainable output. The pipeline runs **after** dedup and **before** delivery. Users should be able to express topic preferences ("I like AI and philosophy videos"), security rules ("reject links from sketchy domains"), and view **why** a specific item was accepted or rejected.

Decision

Stage model ? four ordered tiers

The pipeline is split into four tiers, ordered by cost (cheapest first). Later stages only run if all earlier stages pass:

```

...
????????????????????????????????????????????????????????????
?                               Policy Pipeline                       ?
?                               ?                                     ?
? Tier 1 ? Metadata gates (always runs, zero I/O)                   ?
?   duration, resolution, size, codec, mime, GIF/round              ?
?   ? extends today's FilterEngine                                  ?
?                               ?                                     ?
? Tier 2 ? Topic preferences (opt-in, metadata-only)                ?
?   source allowlist, keyword match, channel topic tags            ?
?   ? user-expressed interests filter the stream                   ?
?                               ?                                     ?
? Tier 3 ? Content safety (opt-in, metadata + light I/O)            ?
?   URL/domain reputation, file metadata sanity,                   ?
?   caption text scanning                                           ?
?   ? cheap checks; no download                                     ?
?                               ?                                     ?
? Tier 4 ? Deep inspection (opt-in, heavy I/O)                      ?
?   ffprobe, perceptual hash, downloaded content scan              ?
?   ? gated behind explicit user opt-in + budget                   ?
????????????????????????????????????????????????????????????
...

```

Stage interface ? `PolicyStage`

Each stage is a pure function (or async function for I/O stages) that takes a `PolicyContext` and returns a `StageResult`:

```

```python
@dataclass
class PolicyContext:
 """Immutable snapshot of what the stage can inspect."""
 content_item: ContentItem # from ADR 0011
 source: Source # the source it came from
 policy: Policy # the owning policy config

@dataclass
class StageResult:
 passed: bool
 reason: str # human-readable explanation
 stage_name: str # e.g. "duration-gate", "url-reputation"

```

```

 metadata: dict[str, object] = field(default_factory=dict) # for explainability UI

class PolicyStage(Protocol):
 """A single gate in the pipeline. Stateless; may be async for I/O stages."""
 async def evaluate(self, ctx: PolicyContext) -> StageResult: ...

```

The pipeline evaluates stages in order and **short-circuits** on the first failure (don't waste I/O on an already-rejected item). The accumulated `StageResult` list is the **explanation trace** ? every decision is auditable.

## Topic preferences ? Tier 2

Topic preferences are stored as a JSON array in `policies.config_json` under a `topics` key:

```

```json
{
  "min_duration_s": 5,
  "topics": {
    "include": ["AI", "Philosophy", "Soccer"],
    "exclude": ["Politics", "Crypto"],
    "mode": "any" // "any" = match any include topic; "all" = match all
  }
}
```

```

Topic matching is **metadata-only** in v1: source title substring match, message caption keyword match, and (future) channel topic tags from Telegram's channel metadata. No ML classification in the design ? that's a separate issue.

## Security stages ? Tier 3

Security stages are **opt-in** and configured per-policy. The design defines three initial stages (implementation deferred to separate issues):

1. **Source allowlist**: reject items from sources not in the `allowlist` (an explicit list of trusted `provider_source_ids`).
2. **URL/domain reputation**: check links in captions against a configurable blocklist. Metadata-only ? no page fetch.
3. **Duplicate suppression**: enhanced dedup that compares by `content_fingerprint` across accounts (already handled by `content_items.dedup_key`; this stage adds cross-policy duplicate detection within an account's delivery streams).

Each security stage records its decision in `StageResult.metadata` for explainability (e.g. `{"blocked_domain": "sketchy-site.com"}`).

## Deep inspection ? Tier 4

Deep inspection (ffprobe, perceptual hash, downloaded content scan) is **gated behind explicit opt-in** because it violates ADR 0003's no-download principle. The opt-in is per-policy (`deep_inspect: true` in `config_json`) and the stage has a **budget** (max bytes per item, max concurrent downloads) enforced by the policy engine.

This tier is explicitly deferred to a follow-up issue ? the design only **reserves the slot** in the pipeline model so it doesn't require a schema migration when implemented.

## Policy evaluation flow (post-dedup, pre-delivery)

```

```python
async def evaluate_pipeline(
    ctx: PolicyContext,
    stages: list[PolicyStage],
) -> tuple[bool, list[StageResult]]:
    results: list[StageResult] = []
    for stage in stages:
        result = await stage.evaluate(ctx)
        results.append(result)
        if not result.passed:
            return False, results # short-circuit
    return True, results
```

```

The caller (telethon\_client or future Bot API handler) records the results via `delivery\_attempts.last\_error` (for failed items) or a new `policy\_decisions` table (for explainability ? deferred to implementation).

## Schema ? minimal changes

**\*\*No schema migration\*\*** is required for this design. The `policies` table already holds a `config\_json` JSON blob. The pipeline is a code concept:

- `policies.config\_json` gains optional `topics`, `allowlist`, `security\_checks`, `deep\_inspect` keys ? all backwards-compatible with the existing `FilterConfig` Pydantic model (extra fields are ignored).
- Future `policy\_decisions` table (deferred) would store the explanation trace for debugging: `(content\_item\_id, policy\_id, stage\_name, passed, reason, metadata\_json, evaluated\_at)`.

## Implementation phasing (future issues)

| Phase         | Scope                   | Dependencies                                                                  |
|---------------|-------------------------|-------------------------------------------------------------------------------|
| <b>**F1**</b> | Policy engine prototype | `PolicyStage` protocol, `evaluate_pipeline()`, wire into live-message handler |
| <b>**F2**</b> | Topic preferences       | `topics` config parsing, source title/caption keyword matching                |
| <b>**F3**</b> | Source allowlist        | `allowlist` config, skip non-allowlisted sources                              |
| <b>**F4**</b> | URL/domain reputation   | Domain blacklist, caption link extraction                                     |
| <b>**F5**</b> | Deep inspection         | ffprobe/perceptual hash, download budget enforcement                          |
| <b>**F6**</b> | Policy decisions table  | Audit trail storage, explainability UI                                        |
| <b>**F7**</b> | Explainability endpoint | `/why <content_item_id>` command showing the explanation trace                |

Each phase is a separate, mergeable PR ? none are bundled here. This ADR only defines the **\*\*design contract\*\*** the phases code against.

## Consequences

### **\*\*Positive\*\***

- Unifies topic, security, and quality gates into a single explainable framework ? no ad-hoc if-chains scattered across handlers.
- Cheap gates always run first; expensive enrichment is only reached for items that pass the cheap gates.
- `StageResult` list provides an audit trail (explainability) without additional DB lookups at decision time.
- Backwards-compatible: `policies.config\_json` already exists; adding optional keys requires no migration.
- Each implementation phase is independently shippable behind feature flags.

### **\*\*Negative\*\***

- Four tiers add conceptual complexity vs. today's flat filter.
- `PolicyStage` protocol requires all stages to conform to the same interface, which may be awkward for deeply different checks (metadata-only vs. I/O-heavy). Mitigated by the `metadata` dict catch-all.
- Topic matching is string-based substring matching in Phase 1 ? not semantic. A future ML stage would replace it, but it's good enough for a v1.
- Deep inspection violates ADR 0003 and must be explicitly opt-in with a budget to avoid runaway costs.

## Alternatives considered

- **\*\*Do nothing; keep flat FilterConfig.\*\*** Rejected ? the umbrella exists to dissolve the conflation of quality, topic, and security into a single boolean. A pipeline is the natural generalization.
- **\*\*Inline each stage in the handler\*\*** (no protocol). Rejected ? makes it impossible to test stages independently, reorder them, or add new ones without touching the hot path.
- **\*\*Use a rules engine / DSL.\*\*** Rejected ? over-engineered for <100 users. A Python list of callables is simpler, testable, and introspectable.
- **\*\*Bundle all phases into a single PR.\*\*** Rejected ? scope discipline. Each phase is independently shippable; this ADR defines the shared interface they code against.

## Open questions for the decision owner

1. **\*\*`StageResult.metadata` shape\*\*** ? dict or typed per-stage? Recommend: dict for Phase 1 (flexible), typed in Phase 2 (once the shape stabilizes).
2. **\*\*Policy decisions table\*\*** ? store every decision, or only failures? Recommend: every decision (explainability), with a TTL-based cleanup for old traces.
3. **\*\*Topic matching\*\*** ? substring only, or regex/glob? Recommend: substring (case-insensitive) for v1; natural-language matching in a follow-up.
4. **\*\*Deep inspection budget\*\*** ? per-item byte limit or per-account bandwidth cap? Recommend: per-item limit (10 MB default) with a per-account monthly cap enforced at the app level.

## References

- ADR 0003 ? metadata-first probing (the no-download principle that Tier 4 must opt in to violate).
- ADR 0009 ? selective channel scanning (the `topics.allowlist` concept generalizes `sources.watched`).
- ADR 0011 ? provider-neutral domain schema (the `policies` table this design extends, `ContentItem` as the `PolicyContext` input).
- Issue #31 ? Track F (this ADR's tracking issue).
- Issue #25 ? umbrella; this is the final track.

### 4. assistant (2026-07-06T19:18:28.874Z)

Now let me verify the key claims: the AND-only short-circuit pipeline vs the Tier 2 topic semantics (include/exclude, mode any/all) and Tier 3 allowlist. Let me check the code claims about FilterConfig and ContentItem too.

### 5. user (2026-07-06T19:18:30.486Z)

"""Pydantic data models for users, channels, videos, and filter config.

These models are the in-memory representation used by the repository layer

(`db.py`) and the filter engine (`filters/`). They are deliberately decoupled from Telethon's own types so the filter engine stays pure and unit-testable without a live Telegram connection.

```
from __future__ import annotations

from datetime import UTC, datetime
from enum import Enum

from pydantic import BaseModel, ConfigDict, Field, field_validator, model_validator

def _utcnow() -> datetime:
 """Timezone-aware UTC 'now', used as a default factory."""
 return datetime.now(UTC)

class DeliveryMode(str, Enum):
 """How a matched video is delivered to the user.

 - ``FORWARD``: forward the video itself to Saved Messages (re-uploads).
 - ``LINK``: send a text message with a t.me link + metadata summary.
 - ``BOTH``: forward the video AND send the link message.
 """

 FORWARD = "forward"
 LINK = "link"
 BOTH = "both"

class Account(BaseModel):
 """A product identity ? the owner of config, channels, routes, and content.

 Decoupled from any specific provider's session (ADR 0010). ``telegram_id``
 is the bridge used by the Bot API front-door (`from.id`) and by the
 legacy userbot path; it is nullable so future providers that don't have a
 Telegram id (e.g. WhatsApp) can still own an ``Account``.
 """

 model_config = ConfigDict(frozen=True)

 id: int = Field(description="Stable surrogate PK; the owner key for all downstream tables")
 telegram_id: int | None = Field(
 default=None,
 description="Telegram user id (bot from.id OR userbot account id). UNIQUE when set.",
)
 provider: str = Field(default="telegram", description="Provider namespace; future:
'whatsapp'")
 display_name: str | None = None
 created_at: datetime = Field(default_factory=_utcnow)

class TelegramSession(BaseModel):
 """An optional MProto (Telethon) session linked to an :class:`Account`.

 A bot-only user has an ``Account`` row but no ``TelegramSession`` row;
 that is what makes them structurally invisible to ``manager.run``
 (ADR 0010). ``session_path`` is data-driven so the file-location
 convention is explicit rather than derived from the PK at runtime.
 """

 model_config = ConfigDict(frozen=True)

 account_id: int = Field(description="FK ? accounts.id; PK (one session per account in v1)")
```

```

 phone: str | None = Field(default=None, description="E.164 phone; used only for bootstrap")
 session_path: str = Field(description="File-backed Telethon session path
(user_<id>.session)")
 enabled: bool = Field(
 default=True,
 description="If False, the manager skips this session (not the account)",
)
 created_at: datetime = Field(default_factory=_utcnow)
 last_login_at: datetime | None = None

```

```

class User(BaseModel):
 """Deprecated: legacy v1 model conflating identity + MTPROTO session.

 Retained as a backwards-compatible view over the new ``accounts`` +
 ``telegram_sessions`` tables (ADR 0010). New code should use
 :class:`Account` + :class:`TelegramSession` directly. The ``db`` layer
 maps ``User`` ? (``Account``, ``TelegramSession``) so existing callers
 and tests keep working without a flag day.
 """

 model_config = ConfigDict(frozen=True)

 id: int = Field(description="Telegram user id (== accounts.telegram_id for legacy rows)")
 phone: str | None = Field(default=None, description="E.164 phone; used only for bootstrap")
 enabled: bool = Field(default=True, description="If False, the manager skips this session")
 created_at: datetime = Field(default_factory=_utcnow)

```

```

class Channel(BaseModel):
 """A channel/group a user is a member of. Populated lazily as messages arrive.

 Deprecated: a provider-neutral view over :class:`Source` (ADR 0011). New
 code should use :class:`Source` directly; the ``db`` layer maps
 ``Channel`` ? ``Source`` so existing callers and tests keep working.
 """

 model_config = ConfigDict(frozen=True)

 id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
 user_id: int
 channel_id: int = Field(description="Telegram channel/chat id")
 title: str | None = None
 seen_first_at: datetime = Field(default_factory=_utcnow)

```

### --- provider-neutral domain models (ADR 0011, Track B) -----

#

**These generalize the legacy v1 ``Channel`` / ``VideoRecord`` / ``FilterConfig`` shapes so the schema no longer leaks Telegram-specific ids into every future provider's code path. ``provider`` columns + provider-prefixed ``dedup\_key`` make cross-provider collisions structurally impossible. See ADR 0011 for the full rationale and the two decisions (Option 1A: tiered ``dedup\_key``; Option 2A: ``policies`` generalizes ``filter\_config``).**

```

class Source(BaseModel):
 """A configured place we ingest content from (ADR 0011).

 Generalizes the legacy :class:`Channel` (1:1 for Telegram today; future

```



```
providers add their own source kinds). ``provider_source_id`` is the
provider-native id (Telegram ``channel_id``, future WhatsApp group id, ?).
"""
```

```
model_config = ConfigDict(frozen=True)
```

```
id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
account_id: int = Field(description="FK ? accounts.id (owner of the source)")
provider: str = Field(default="telegram", description="Provider namespace")
provider_source_id: str = Field(
 description="Provider-native source id (Telegram channel_id, ?)"
)
title: str | None = None
kind: str = Field(default="channel", description="'channel' | 'group' | 'feed'")
watched: bool = Field(
 default=False,
 description="ADR 0009 selective scanning: ingest from this source iff True",
)
seen_first_at: datetime = Field(default_factory=_utcnow)
last_seen_at: datetime | None = None
```

```
class ContentItem(BaseModel):
```

```
 """A normalized content record discovered from any :class:`Source` (ADR 0011).
```

```
 Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
 (``content_type='video'``); ``text`` / ``link_url`` are landed now but stay
 NULL until later tracks populate them (avoids a future ``ALTER TABLE``).
 ``dedup_key`` is the provider-neutral dedup key (Option 1A: tiered lookup
 wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
 """
```

```
model_config = ConfigDict(frozen=True)
```

```
id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
account_id: int = Field(description="FK ? accounts.id")
source_id: int = Field(description="FK ? sources.id")
provider: str = Field(default="telegram", description="Provider namespace")
provider_message_id: str = Field(
 description="Provider-native message id (Telegram message_id, ?)"
)
content_type: str = Field(default="video", description="'video' | 'link' | 'article'")
text: str | None = Field(default=None, description="Caption / post text (nullable in Phase
1)")
link_url: str | None = Field(default=None, description="For link/article posts (Phase 1:
NULL)")
media_ref: str | None = Field(default=None, description="Provider media reference")
document_id: str | None = Field(
 default=None,
 description="Telegram document.id (kept for TG-specific Tier-1 lookups)",
)
duration_s: int | None = None
width: int | None = None
height: int | None = None
size_bytes: int | None = None
mime_type: str | None = None
dedup_key: str = Field(description="Provider-neutral dedup key (Option 1A); unique per
account")
matched: bool = False
seen_at: datetime = Field(default_factory=_utcnow)
```

```
class Policy(BaseModel):
```

```
 """A named, reusable filtering policy owned by an account (ADR 0011, Option 2A).
```

```

Generalizes the legacy per-account ``filter_config`` row so one account can
own multiple policies (enables ADR 0009's per-route filter overrides) and so
future enrichment/security stages grow into the same ``config_json`` column.
Exactly one policy per account carries ``is_default=True``; the legacy
``load_filter_config`` / ``save_filter_config`` shims read/write that row.
"""

```

```

model_config = ConfigDict(frozen=True)

id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
account_id: int = Field(description="FK ? accounts.id")
name: str = Field(description="Policy name; 'default' for the account default policy")

```

## 6. assistant (2026-07-06T19:18:33.986Z)

The finding notes ContentItem carries `text` (caption) but is NULL in Phase 1. Let me check the FilterConfig model for extra-key handling, and confirm the pipeline/topic semantics claim is the core of the finding.

## 7. user (2026-07-06T19:18:35.958Z)

```

name: str = Field(description="Policy name; 'default' for the account default policy")
config_json: str = Field(
 description="FilterConfig JSON blob (Phase-1 shape); grows in Phase 2 (Track F)"
)
is_default: bool = Field(
 default=False, description="Exactly one per account (UNIQUE partial index)"
)
created_at: datetime = Field(default_factory=utcnow)
updated_at: datetime = Field(default_factory=utcnow)

```

## --- Track C: destinations, routing, delivery state (ADR 0011) -----

#

**These tables were created empty by Track B (\_SCHEMA); Track C populates destinations (backfilled from filter\_config\_deprecated.config\_json), writes delivery\_attempts (replacing the legacy videos.forwarded boolean), and defines the models for per-destination delivery state. The source/destination invariant (a channel is either source or destination, never both) is enforced at the app level by future selective-scanning logic (Track E, #30).**

```

class Destination(BaseModel):
 """A delivery target ? where matched content is sent (ADR 0011, Track C).

 Generalizes the legacy ``FilterConfig.delivery_target`` field so one account
 can have multiple named output channels. Backfilled from the existing
 per-account ``delivery_target``; the default destination is the target used
 when no explicit routing rule matches.
 """

 model_config = ConfigDict(frozen=True)

 id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
 account_id: int = Field(description="FK ? accounts.id")
 provider: str = Field(default="telegram", description="Provider namespace")
 target: str = Field(description="Telethon entity: 'me', '@username', or chat id")
 kind: str = Field(default="saved", description="'saved' | 'channel' | 'chat'")
 title: str | None = None

```

```

is_output: bool = Field(
 default=False,
 description="ADR 0009: distinguishes output channels from ingest sources",
)
created_at: datetime = Field(default_factory=_utcnow)

class RoutingRule(BaseModel):
 """A rule that maps a source through a policy to a destination (ADR 0009).

 Canonical name: ``routing_rules`` (locked in umbrella issue #25). When
 ``policy_id`` is NULL, the account's default policy is used. When multiple
 rules match, the highest ``priority`` wins.
 """

 model_config = ConfigDict(frozen=True)

 id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
 account_id: int = Field(description="FK ? accounts.id")
 source_id: int = Field(description="FK ? sources.id")
 policy_id: int | None = Field(
 default=None,
 description="FK ? policies.id; NULL means use account default policy",
)
 destination_id: int = Field(description="FK ? destinations.id")
 enabled: bool = Field(default=True)
 priority: int = Field(default=0, description="Higher wins when multiple rules match")
 created_at: datetime = Field(default_factory=_utcnow)

class DeliveryStatus(str, Enum):
 """Per-destination delivery state, replacing the legacy binary ``forwarded``."""

 PENDING = "pending"
 DELIVERED = "delivered"
 FAILED = "failed"
 RETRYING = "retrying"

class DeliveryAttempt(BaseModel):
 """Per-destination delivery record ? idempotent via UNIQUE index.

 Replaces the legacy ``videos.forwarded`` boolean (ADR 0011). One row per
 ``(content_item_id, destination_id)`` pair. On retry, ``attempt_count`` is
 incremented and ``status``/``last_error`` are updated.
 """

 model_config = ConfigDict(frozen=True)

 id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
 content_item_id: int = Field(description="FK ? content_items.id")
 destination_id: int = Field(description="FK ? destinations.id")
 route_id: int | None = Field(default=None, description="FK ? routing_rules.id (nullable)")
 status: DeliveryStatus = Field(default=DeliveryStatus.PENDING)
 attempt_count: int = Field(default=0, description="Incremented on each retry")
 last_error: str | None = None
 created_at: datetime = Field(default_factory=_utcnow)
 delivered_at: datetime | None = None
 next_retry_at: datetime | None = None

class FilterConfig(BaseModel):
 """Per-user filter configuration.

 v1 rules: duration bounds, minimum height (resolution), and max file size.

```

Stored in the DB as a JSON blob (``filter\_config.config\_json``) so adding a new field later is backwards-compatible.

"""

# Duration bounds in seconds. None means unbounded on that side.

min\_duration\_s: int | None = Field(default=5, ge=0)

max\_duration\_s: int | None = Field(default=7200, ge=0)

# Resolution: a video passes if its height >= target\_height.

# None disables the resolution rule.

target\_height: int | None = Field(default=1080, ge=1)

# Max file size in mebibytes. None disables the size rule.

max\_size\_mb: int | None = Field(default=2048, ge=1)

# Content-type exclusions. GIFs and round videos are excluded by default

# (their duration/dimensions are usually incidental to real video filters);

# silent videos are off by default since some legit clips have no audio.

## 8. assistant (2026-07-06T19:19:00.135Z)

Now I have full context. The finding is a design-critique about expressiveness. Let me analyze the core claim carefully.

The finding claims: the pipeline is pure AND with first-failure short-circuit (`evaluate_pipeline` returns False on `not result.passed`), but Tier 2 topics semantics (include OR/any-or-all, exclude veto) and Tier 3 allowlist cannot be expressed as an ordered AND chain of independent per-topic pass/fail stages ? they must collapse into monolithic stages, and precedence between exclude/include/allowlist is undefined.

Let me adversarially test whether the ADR actually addresses this.

Key question 1: Does the ADR say each topic is its own stage, or that topics is one stage?

Looking at the phasing table: "F2 ? Topic preferences: `topics` config parsing, source title/caption keyword matching". And the config is a single `topics` object with include/exclude/mode. Nothing in the ADR claims one-stage-per-topic. The pipeline diagram lists "Tier 2 ? Topic preferences" as a tier containing "source allowlist, keyword match, channel topic tags" ? these are described as the tier's contents, not as multiple independent AND stages.

Let me re-read the exact stage-composability claims.

The ADR's alternatives section: "Inline each stage in the handler (no protocol). Rejected ? makes it impossible to test stages independently, reorder them, or add new ones." So the composability claim (independent test/reorder/add) is about *stages*, and the ADR does not commit to topic-per-stage granularity. A single "topic-preferences" stage doing internal include-OR/exclude-veto is fully consistent with the AND-chain-of-stages model ? that stage returns one `StageResult(passed=...)`.

So the finding's strongest sub-claim ("a chain of per-topic AND stages cannot represent OR") attacks a design the ADR never states. The ADR treats Tier 2 as (at minimum) one stage. Within-stage boolean logic is exactly what a stage is for.

However ? the finding has a **second, real** sub-claim: precedence/interaction between exclude, include (mode any/all), and Tier-3 allowlist is genuinely **undefined** in the ADR. Let me verify that precisely.

- Topics: `include` + `exclude` + `mode: any/all`. The ADR says mode "any = match any include topic; all = match all". It does **not** say how `exclude` combines with `include` (does an exclude hit veto even if include matches? what if a video matches both an include and exclude topic?).

- Tier 3 "Source allowlist: reject items from sources not in the allowlist." vs Tier 2 also mentions "source allowlist" in the diagram. And References says "topics.allowlist generalizes sources.watched." There's an ambiguity: allowlist appears in BOTH the Tier 2 diagram box AND as a Tier 3 security stage. Whether allowlist bypasses topic filtering is undefined.

Let me confirm the allowlist appears in both places.

Tier 2 diagram box: "source allowlist, keyword match, channel topic tags". Tier 3 security stage #1: "Source allowlist: reject items from sources not in the allowlist". So allowlist is duplicated across tiers 2 and 3, and its interaction/precedence with topic include/exclude is unspecified. This supports the finding's "allowlist-vs-topic interactions undefined."

Now, is this a *high* or *medium* issue? The finding self-rates medium. Under the calibration (false claim/data-loss = high; unspecified interaction/gap = medium), this is an unspecified-interaction gap in a Proposed draft design doc. That's squarely medium.

Now the adversarial REFUTE angle: Is the "AND-only cannot express OR" framing an inaccuracy that undermines the finding? Partly ? the ADR never claims topic-per-stage, so the "chain of per-topic AND stages cannot represent OR" is a strawman. But the finding's *core* survives: the ADR does not specify the within-tier boolean semantics (exclude vs include precedence, allowlist vs topics), and the mode: any OR-semantics are only defined for includes, silent on how exclude and allowlist compose. That is a genuine, real completeness gap in the design contract.

Let me double check there is truly no precedence statement anywhere in the ADR.

## 9. user (2026-07-06T19:19:02.554Z)

```
25:| `FilterConfig` / `Policy` | One JSON blob per account. Flat
duration/resolution/size/exclude rules. AND semantics. |
55:? source allowlist, keyword match, channel topic tags ?
109: "exclude": ["Politics", "Crypto"],
125:1. **Source allowlist**: reject items from sources not in the `allowlist`
174:- `policies.config_json` gains optional `topics`, `allowlist`,
187:| **F3** ? Source allowlist | `allowlist` config, skip non-allowlisted sources | F1 |
256:- ADR 0009 ? selective channel scanning (the `topics.allowlist` concept
```

## 10. assistant (2026-07-06T19:19:19.334Z)

Confirmed. The ADR mentions `exclude` and `allowlist` in config examples and stage lists, but there is **no statement anywhere** defining:

1. How `exclude` combines with `include` (does exclude veto override an include match?).
2. How `mode: any/all` interacts with `exclude` (mode is only defined for includes).
3. How the allowlist relates to topics ? and note allowlist is duplicated: it appears in the Tier 2 diagram box (line 55) AND as Tier 3 security stage #1 (line 125) AND as `topics.allowlist` in References (line 256), with no single home or precedence rule.

Now let me assess the finding's accuracy holistically for the verdict.

**\*\*What holds (CONFIRMED core):\*\***

- The pipeline IS strict AND with first-failure short-circuit ? verified: `if not result.passed: return False, results`. No OR/veto/score/priority verdict. `StageResult.passed` is a bare bool.
- The within-tier boolean semantics ARE underspecified: exclude-vs-include precedence, mode: any/all applying only to includes, allowlist-vs-topics interaction are all genuinely undefined. This is a real completeness gap in a design contract.
- The Negative section admits interface awkwardness ("may be awkward for deeply different checks") but does NOT acknowledge the boolean-expressiveness / within-tier-composition limit. Verified.

**\*\*What's overstated (the REFUTE pressure):\*\***

- "a chain of per-topic AND stages cannot represent OR ... it must collapse into one monolithic topic stage ? at which point the abstraction isn't buying the composability the ADR claims (independent test/reorder/add)." This is the weak part. The ADR never proposes per-topic stages; the composability claim is about *stages* (F2 is a single topic stage), and a single stage doing internal OR is entirely consistent with the model and still independently testable/reorderable. So the "abstraction isn't buying composability" conclusion is not established ? a composite topic stage is a legitimate stage. The ADR's composability claim is not actually contradicted.

So the finding is partially inaccurate in its framing (the "monolithic stage defeats composability" argument doesn't land) but its **\*\*substantive recommendation is valid and its core gap is real\*\***: the ADR should specify within-tier boolean semantics and the

exclude/include/allowlist precedence order. The recommendation ("specify how within-tier boolean logic is handled ... state the precedence order between exclude, include, and allowlist") is exactly the right fix.

Verdict: CONFIRMED ? but calibrated. The genuine defect is the **undefined precedence/interaction semantics** (exclude vs include, mode scope, allowlist vs topics), not the "AND-only can't express OR" theoretical claim which is partly a strawman since a stage may contain internal boolean logic. Severity: medium ? it's an unspecified-interaction gap in a Proposed draft, would force F2/F3 implementers to invent semantics not in the contract, but no false claim and no data loss.

#### **11. user (2026-07-06T19:19:34.676Z)**

Structured output provided successfully